

Database Scaling — Practical System Design Notes

Hinglish • Common-sense • Interview-focused • Core concepts + Q&A;

1. Database Bottleneck Kya Hota Hai?

Database bottleneck tab hota hai jab application ki demand database ki processing capacity se zyada ho jaye. Node.js aur Load Balancer fast hone ke baad bhi slow DB poore system ki latency badha sakta hai.

```
10,000 requests/sec
    ↓
Node.js × N
    ↓
Database
    ↓
Capacity = 2,000 q/s
```

Result:
queue + latency + timeouts + errors

Interview mental model: Pehle bottleneck identify karo; blindly scaling mat karo.

2. First Debugging Step — Query Check

Database slow hone par pehle expensive queries identify karo. Query plan, execution time, rows scanned, joins, sorting, locking, and frequency dekho.

```
Request
  ↓
API
  ↓
Slow SQL query
  ↓
DB scans too many rows
  ↓
high CPU / I/O
  ↓
slow response
```

Question: Agar ek API 2 sec le rahi hai aur DB query 1.8 sec le rahi hai, toh first suspect kya hoga?

Database query/path ko investigate karna logical first step hai, rather than immediately adding more Node.js servers.

3. Indexing

Index database ko relevant rows jaldi locate karne mein help karta hai. Simple analogy: book ke index ke bina page search karna vs index se directly page locate karna.

Without index:
`SELECT * FROM users WHERE email = 'a@x.com';`

DB → many rows scan → find match

With suitable index:
email index → locate matching row faster

Index read performance improve kar sakta hai, but index bhi storage leta hai aur writes/updates ko extra work de sakta hai. Isliye har column par blindly index nahi lagana chahiye.

4. Composite Index

Multiple columns ko together index kiya ja sakta hai. Column order matters because query patterns and index structure determine how effectively the index can be used.

Query:

```
WHERE customer_id = 10  
      AND created_at > '2026-01-01'
```

Possible composite index:

(customer_id, created_at)

Think:

first narrow by customer_id,
then use created_at range.

Interview point: Composite index ka order query pattern ke according choose hota hai; 'more columns = always better' nahi hota.

5. EXPLAIN / Query Plan

EXPLAIN-type tools se database bata sakta hai ki query ko kaise execute karne ka plan hai—index use ho raha hai ya full scan, estimated/actual rows, joins, sorting, etc.

```
EXPLAIN
↓
query plan
  ■→ index scan?
  ■→ table scan?
  ■→ join strategy?
  ■→ rows examined?
```

Production debugging mein query timing + execution plan + DB CPU/I/O/locks ko together dekho.

6. Connection Pool

Har request par naya DB connection create karna expensive ho sakta hai. Connection pool existing connections ko reuse karta hai.

```
Node.js
↓
Connection Pool
  ■→ DB connection 1
  ■→ DB connection 2
  ■→ DB connection 3
  ■→ DB connection N
```

Request comes → borrow
Query runs → release back to pool

Pool size bahut chhota hua toh requests wait karenge; bahut bada hua toh DB par too many concurrent connections ka pressure aa sakta hai.

7. Vertical Scaling

Database server ko bigger machine dena: more CPU, RAM, faster storage, etc. Ye simple hai but hardware limits aur cost ke constraints hote hain.

```
DB Server
Small
↓
More CPU + RAM + faster disk
↓
Bigger DB Server
```

Pros: relatively simple. **Cons:** physical/resource ceiling and potentially larger single-machine dependency.

8. Read Replicas

Agar application mein reads bahut zyada hain, primary database ke saath read replicas use ki ja sakti hain. Writes generally primary par jaati hain; reads replicas par distribute ki ja sakti hain, subject to consistency requirements.

```
Writes → Primary ■
          ■→ Read Replica 1
          ■→ Read Replica 2
          ■→ Read Replica 3
```

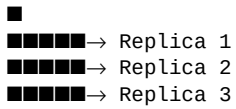
Read traffic → replicas
Write traffic → primary

Important: replication lag ho sakta hai. Write ke immediately baad replica par read karne se old value mil sakti hai.

9. Replication

Replication ka matlab database state ko other database instances par copy karna. It can improve read scale and availability depending on topology.

Primary



Replication backup ke equal nahi hai. Primary par accidental delete/corruption replicate ho sakta hai; independent backups/recovery strategy still important hai.

10. Replication Lag

Primary par write aur replica par us write ke visible hone ke beech delay ko replication lag bol sakte hain.

Client



WRITE → Primary



immediately READ → Replica



may see old value

After replication catches up:

READ → new value

Mitigation depends on requirement: read-after-write ko primary se serve karna, session/transaction routing, synchronous replication where supported/appropriate, or application-level consistency handling.

11. Read/Write Splitting

Application ya DB proxy writes ko primary aur reads ko replicas ki taraf route kar sakta hai.

```
Node.js
■■■ WRITE ■■→ Primary
■■■ READ  ■■→ Replica pool
```

But every read replica par bhejna correct nahi. Strong consistency ya read-after-write requirements ko understand karna zaroori hai.

12. Horizontal Scaling

Horizontal scaling mein workload ko multiple machines/partitions mein distribute kiya jata hai. Database mein commonly replicas aur sharding is direction mein use hote hain.

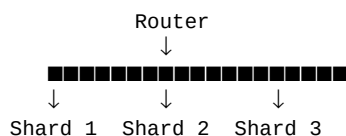
```
One DB
↓
Multiple DB nodes / shards

Shard A → customers 1-1M
Shard B → customers 1M-2M
Shard C → customers 2M-3M
```

Horizontal scaling powerful hai, but routing, joins, transactions, rebalancing and operational complexity badh sakti hai.

13. Sharding

Sharding means data ko multiple database nodes par partition karna. Example: customer_id ke hash/range ke basis par records different shards par ja sakte hain.



Different keys → different shards

Sharding tab consider karo jab single database ki capacity/throughput ko replicas ya vertical scaling se sufficiently solve nahi kiya ja raha.

14. Shard Key

Shard key decide karti hai ki record kis shard par jayega. Good shard key distribution even rakhti hai aur common queries ko efficiently route karne deti hai.

Bad shard key se hotspots ho sakte hain. Example: monotonically increasing key ka naive range partitioning ek shard ko current writes ka disproportionate load de sakta hai.

15. Partitioning vs Sharding

Concept	Simple meaning
Partitioning	One logical database/table ke data ko partitions mein organize/split karna.
Sharding	Data ko multiple independent database nodes/shards par distribute karna.
Common goal	Large data/workload ko manageable pieces mein divide karna.

Terminology database technology ke according vary kar sakti hai, so interview mein exact product/context clarify karna useful hai.

16. Transactions

Transaction multiple DB operations ko logical unit ke roop mein treat karti hai. Banking-style money transfer classic example hai: debit aur credit ko consistent manner mein handle karna.

```
BEGIN
  debit A
  credit B
COMMIT
```

If required operation fails:
ROLLBACK

Interview mein ACID explain karna useful hai: Atomicity, Consistency, Isolation, Durability.

17. Database Locks / Contention

Concurrent transactions same rows/data ko modify karte waqt locks ya contention create kar sakti hain. Excessive lock waiting latency aur throughput ko hurt kar sakta hai.

```
Transaction A → locks row X
Transaction B → needs row X
                ↓
                waits
                ↓
            higher latency
```

Production investigation mein slow queries ke saath lock waits, deadlocks, transaction duration, and isolation behavior bhi inspect karo.

18. Database Connection Pool + Multiple Node Servers

Important interview trap: 10 Node servers × 100 DB connections = potentially 1,000 concurrent DB connections. Horizontal scaling application ko easy lag sakta hai, but DB connection capacity hit ho sakti hai.

```
10 Node instances
x
100 pool connections each
=
up to 1,000 DB connections
```

Pool size ko application traffic aur DB capacity ke saath tune karo. DB ko overwhelm karne ke liye unlimited connections solution nahi hain.

19. Caching + Database

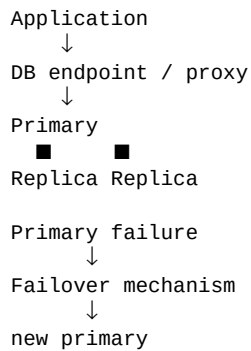
Redis database reads ko reduce kar sakta hai, but cache correctness important hai. A typical architecture:

```
Request
↓
Redis
■ → HIT → response
■
■ → MISS → DB
                ↓
            Redis SET
                ↓
            response
```

Caching DB bottleneck ko reduce karta hai, but bad cache keys, stale data, stampedes, and hot keys new bottlenecks create kar sakte hain.

20. Database Failure & Failover

Production DB architecture mein failure handling important hai: health detection, replica promotion/failover, connection retry strategy, backups, restore testing, and application behavior during partial outage.



Failover ka matlab sirf 'replica exists' nahi; application ko correct endpoint, retry behavior, transaction safety, and recovery process bhi chahiye.

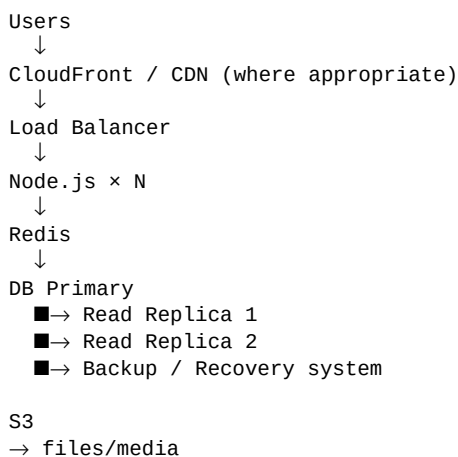
21. Database Bottleneck Debugging Checklist

1. API latency measure karo: p50 / p95 / p99.
2. Slowest DB queries identify karo.
3. Query execution plan / EXPLAIN dekho.
4. Index usage and missing/unused indexes inspect karo.
5. Rows scanned vs rows returned compare karo.
6. DB CPU, RAM, disk I/O, storage latency check karo.
7. Connection pool saturation check karo.
8. Lock waits / deadlocks check karo.
9. Read/write ratio samjho.
10. Cache hit rate check karo.
11. Read replicas / replication lag check karo.
12. Only then decide: vertical scale, replicas, partitioning, or sharding.

22. Vertical vs Horizontal vs Read Replicas vs Sharding

Approach	What changes?	Main use
Vertical scaling	Bigger DB machine	Simple capacity increase
Read replicas	Copies for reads	High read workload + availability
Horizontal/sharding	Data/workload across nodes	Very large scale
Caching	Serve repeated data outside DB	Reduce DB reads / latency

23. Real Production Architecture



This is a conceptual architecture. Exact components depend on consistency, traffic pattern, data model, failure requirements, and cloud/database technology.

24. High-Value Interview Questions + Short Answers

Question	Short answer
DB slow ho toh first kya check karoge?	Slow queries, execution plans, indexes, DB resource metrics, locks and connection pool.
Index kya karta hai?	Relevant records locate karne ka work reduce kar sakta hai.
Har column par index kyun nahi?	Storage and write/update overhead; optimizer may not benefit.
Connection pool kya hai?	Reusable DB connections ka managed pool.
Pool too large hua toh?	DB par excessive concurrent connections/load create ho sakta hai.
Vertical scaling?	Same DB server ko more CPU/RAM/faster storage dena.
Read replica?	Primary ki replicated copy, generally read traffic scale/availability ke liye.
Replication lag?	Primary write aur replica visibility ke beech delay.
Read-after-write issue?	Replica old data return kar sakta hai; route critical read to primary or use suitable consistency
Sharding?	Data ko multiple database nodes/shards mein distribute karna.
Good shard key?	Even distribution + efficient routing for common access patterns.
Replication = backup?	No. Replicas can copy accidental changes; independent backups are still needed.
Caching DB bottleneck kaise help karta hai?	Repeated reads Redis se serve karke DB queries reduce karta hai.
Why not unlimited DB connections?	Connections consume DB resources and can reduce throughput rather than improve it.
When shard?	When vertical scaling, query optimization, caching and/or replicas are insufficient for required

25. Final Cheat Sheet

DB Scaling Mental Model

1. Optimize query
↓
2. Add/fix indexes
↓
3. Tune connection pool
↓
4. Add caching
↓
5. Vertical scaling
↓
6. Read replicas
↓
7. Partitioning
↓
8. Sharding

Debug signals:

Latency → p50/p95/p99

CPU → DB compute pressure

I/O → storage pressure

Connections → pool saturation

Locks → contention

Replication lag → consistency/read routing

Cache hit rate → cache effectiveness

Golden rule:

"First identify the bottleneck,
then choose the scaling technique."

26. Practice Scenarios

1. Product API suddenly goes from 100 ms to 2 seconds. How will you debug it?
2. A query scans 10 million rows to return 5 rows. What would you investigate?
3. You have 90% reads and 10% writes. What architecture options could reduce primary DB load?
4. Write succeeds, immediate read returns old data. Why?
5. Ten Node servers each have a 100-connection pool. What could go wrong?
6. Read replicas are overloaded while primary is mostly idle. What would you inspect?
7. A single customer_id receives huge traffic. How can shard-key design create a hotspot?
8. When would you choose vertical scaling before sharding?
9. Why is replication not enough as a backup strategy?
10. Design a production architecture for 10k requests/sec with heavy product reads.
11. How would Redis and read replicas complement each other?
12. What metrics would you put on a DB dashboard?

27. Practical Local Exercise

Build a small Node.js + SQL application and intentionally create a slow query. Measure its latency, add an index, compare execution time, then add Redis cache and compare DB query count. After that, simulate multiple Node instances and tune the connection pool. This gives practical understanding of query optimization, caching and DB scaling.